

# Codebase approach

How this project reads P&ID DWGs and turns them into tagged, geo-located records — then uses those records for hierarchy AI.

**Drawing used in examples:** `inputs/Broke System.dwg` (Shotton PM3, Broke System)

## Overview

|        |                  |                                     |
|--------|------------------|-------------------------------------|
| Step 1 | Read DWG         | → structural CAD model (JSON)       |
| Step 2 | Extract tags+XYZ | → inventory + tag register          |
| Step 3 | Enrich           | → line bindings, loops, title block |
| Step 4 | Hierarchy AI     | → zoomed screenshot + prompt → CSV  |

Steps 1–2 are **deterministic CAD parsing** (no AI). Steps 3–4 add engineer-facing structure and AI hierarchy. All four steps are covered below.

## Step 1 — Read the DWG

**Script:** `dwg_pure_dump.py`

**Command:** `make run-json` (or `make all`)

## Why ODA is required

DWG is a proprietary binary format. **ezdxf** can read DXF directly, but for `.dwg` it uses the **ODA File Converter** addon (`ezdxf.addons.odafc`):

`Broke System.dwg` → ODA File Converter → in-memory ezdxf document

`configure_odafc()` locates the converter (e.g. `~/Applications/ODAFFileConverter.app` or `~/bin/ODAFFileConverter`) and sets `odafc.unix_exec_path`.

# What we parse

Once opened, we walk **modelspace** and extract:

| Object                           | Source                          | Key fields                                           |
|----------------------------------|---------------------------------|------------------------------------------------------|
| <b>INSERT</b> (block references) | P&ID symbols                    | insert point [x,y,z] , layer, block name, attributes |
| <b>TEXT / MTEXT</b>              | Tag labels, line numbers, notes | position [x,y,z] , text, layer                       |
| <b>LINE / LWPOLYLINE</b>         | Piping geometry                 | start/end vertices, layer                            |
| Layers, block defs, layouts      | Drawing structure               | names, counts, hierarchy                             |

## Files written:

```
outputs/Broke System.full_dump.xlsx          # human-readable workbook
outputs/jsons/Broke System.structural_dump.json
```

# Sample output

**Counts** (top of JSON):

```
{
  "inserts": 2058,
  "entities_total": 6808,
  "text_entities": 2441,
  "layers": 38,
  "blocks": 187
}
```

**INSERT** (P&ID symbol at XYZ):

```
{
  "handle": "64302",
  "name": "PPI_1302A-25_0",
  "insert": [660.0, 297.5, 0.0],
  "layer": "P-EQUIPMENT_POS",
  "rotation": 0,
  "attributes": []
}
```

**TEXT** (equipment tag label — separate from the block):

```
{
  "handle": "7AA34",
  "type": "TEXT",
  "text": "35-24L008",
  "layer": "P-EQUIPMENT_POS",
  "position": [665.65, 293.77, 0.0]
}
```

**LINE** (pipe segment geometry):

```
{
  "type": "LINE",
  "layer": "P-FITTINGS",
  "handle": "75F1E",
  "geometry": {
    "start": [655.0, 251.0, 0.0],
    "end": [662.0, 251.0, 0.0]
  }
}
```

## Coordinate system

- Units come from the drawing header (paper/engineering units as stored in DWG).
- Each INSERT has an **insert point** from `entity.dxf.insert` → stored as `[x, y, z]`.
- P&IDs are effectively **2D**; `z` is almost always `0.0` but we keep it for completeness.
- All later distance math (nearest tag, crop window) uses **x/y** in CAD space.

## Step 2 — Extract tags with X, Y, Z

Tags are **not** a single field on the symbol block. On this P&ID standard, the **equipment tag is usually separate TEXT** near the symbol. We therefore do two passes:

1. **Inventory** — place every symbol at its CAD coordinates
2. **Tag register** — bind the nearest label text to each symbol

### 2a. Inventory — symbols at XYZ

**Script:** `dwg_pid_inventory.py`

**Command:** `make inventory`

Reads `structural_dump.json` and classifies each **INSERT** by **layer first**, then block name:

| Layer example   | Category            |
|-----------------|---------------------|
| P-EQUIPMENT_P0S | process_equipment   |
| P-PUMP_P0S      | pumps               |
| P-VALVEP0S      | valves              |
| P-CVP0S         | control_valves      |
| P-INSTRP0S      | instruments         |
| P-LINEP0S       | lines (text labels) |

For each classified insert, `base_component()` copies coordinates:

```
x, y, z = insert[0], insert[1], insert[2] if len(insert) > 2 else 0.0
```

**Files written:**

```
outputs/Broke System.pid_inventory.xlsx
outputs/jsons/Broke System.pid_inventory.json
outputs/jsons/Broke System.pid_validation.json
```

# Sample output

**Bucket counts** (summary from JSON):

|                       |                  |             |
|-----------------------|------------------|-------------|
| process_equipment: 79 | pumps: 55        | valves: 227 |
| control_valves: 94    | instruments: 402 | lines: 215  |
| pipe_segments: 1214   | connections: 599 |             |

**Equipment symbol** (INSERT → XYZ; tag still block name at this stage):

```
{
  "component_type": "process_equipment",
  "block_name": "PPI_1302A-25_0",
  "layer": "P-EQUIPMENT_POS",
  "x": 660.0,
  "y": 297.5,
  "z": 0.0,
  "position": "660.000000,297.500000,0.000000",
  "source": "insert"
}
```

**Line label** (TEXT on P-LINEPOS → parsed fields + XYZ):

```
{
  "component_type": "lines",
  "line_number": "35-24-009-PP-600-E10H2A",
  "plant_area": "35-24",
  "line_sequence": "009",
  "layer": "P-LINEPOS",
  "x": 657.40,
  "y": 251.25,
  "source": "line_label"
}
```

At this stage equipment rows use **block names** ( PPI\_1302A-25\_0 ), not plant tags ( 35-24L008 ) — resolved in Step 3.

## 2b. Tag register — resolve plant tags near symbols

**Script:** `dwg_pid_enrich.py`

**Command:** `make enrich`

`build_tag_register()` takes inventory + structural text and **spatially binds labels**:

Symbol at (x, y) → search TEXT/MTEXT within radius → resolved\_tag

| Search         | Radius        | Layers                                          | Match                           |
|----------------|---------------|-------------------------------------------------|---------------------------------|
| Equipment tags | 90 CAD units  | P-EQUIPMENT_POS , P-PUMP_POS , P-VALVEPOS , ... | 35-24L008 , 35-24P509 , ...     |
| Descriptions   | 120 CAD units | P-TEXT , P-EQUIPMENT_POS , ...                  | WINDER PULPER , 800 BDTPD , ... |

Tag pattern (regex): `^\d{2}-\d{2}[A-Z]\d{2,4}[A-Z]?$` (e.g. 35-24L008 , 35-24P519 ).

### Algorithm (per inventory insert):

1. Take symbol (x, y) from inventory
2. Find nearest matching text entities (sorted by distance)
3. Set `resolved_tag` to the closest equipment-tag match
4. Store `nearby_tags` and `nearby_descriptions` for context

Note: tag register is written in Step 3 ( `make enrich` ); see sample output there.

## 2c. Line numbers and other text tags

Line numbers ( 35-24-009-PP-600-E10H2A ) live as **TEXT on P-LINEPOS** , with their own (x, y) in inventory (Step 2). Step 3 binds them to nearby pipe geometry.

## Step 3 — Enrich (engineer-facing structure)

**Script:** `dwg_pid_enrich.py`

**Command:** `make enrich` (runs after inventory; included in `make all` )

Step 2 gave us **symbols + resolved tags**. Step 3 adds the relationships an engineer cares about — still **no AI**, all rule/spatial logic.

## What enrichment produces

| Extract                 | What it does                                | Used later for                       |
|-------------------------|---------------------------------------------|--------------------------------------|
| Tag register            | Symbol (x,y) → resolved_tag (from Step 2b)  | Parent pick, dossier, zoom           |
| Line geometry bindings  | P-LINEPOS text → nearest pipe segment       | Dossier line short-ids ( 35-24-009 ) |
| Control loop candidates | Instrument ↔ nearest control valve clusters | Context (optional)                   |
| Title block             | SITE, LINE, PROCESS from drawing header     | Prompt plant context                 |
| Revisions / CAD tables  | Revision register, table snapshots          | Audit / metadata                     |

## Line binding (example)

For each line-number text at (x, y) on P-LINEPOS :

1. Parse full line string ( 35-24-009-PP-600-E10H2A )
2. Search nearby pipe segments (LINE / LWPOLYLINE on pipe layers)
3. Pick closest anchor point within ~80 CAD units
4. Store short id + bind confidence

This is why the AI dossier can say 35-24-009 instead of the full pipe spec string.

### Files written:

```
outputs/Broke System.pid_enrichment.xlsx
outputs/jsons/Broke System.pid_enrichment.json
```

Key arrays: tag\_register[] , line\_geometry\_bindings[] , title\_block[] , control\_loop\_summaries[] .

# Sample output

**Tag register** (symbol XYZ → resolved plant tag):

```
{
  "category": "process_equipment",
  "block_name": "PPI_1302A-25_0",
  "handle": "64302",
  "layer": "P-EQUIPMENT_POS",
  "x": 660.0,
  "y": 297.5,
  "resolved_tag": "35-24L008",
  "nearby_tags": "35-24L008; 35-24P509",
  "nearby_descriptions": "WINDER PULPER; HP-33G2; 800 BDTPD",
  "confidence": "high"
}
```

**Line geometry binding** (label text → nearest pipe):

```
{
  "line_number": "35-24-009-PP-600-E10H2A",
  "bind_confidence": "high",
  "bound_pipe_type": "LWPOLYLINE",
  "bind_distance": 1.254,
  "label_position": [657.40, 251.25, 0.0]
}
```

**Title block** (plant context for AI prompt):

```
{
  "PROJECT2": "Shotton Paper Mill, United Kingdom",
  "PROJECT3": "Shotton PM3",
  "TITLE1": "Broke System",
  "SHEET": "1/1",
  "DRAWINGID": "STOD206339.10"
}
```



# Step 4 — Hierarchy AI (screenshot + vision + prompt)

**Script:** `dwg_pid_hierarchy_ai.py`  
**Command:** `make hierarchy-ai TAGS=35-24L008,35-24P509`  
**Defaults:** Claude Sonnet 4.6 + `prompts/pid_hierarchy_gt_v4_dossier.md`

One Bedrock call **per parent tag** (FUNCTION). We do not send the whole drawing.

## 4.1 Pick parent + center

From `tag_register` , select the primary row for the requested tag (e.g. `35-24L008` → `process_equipment` at `(660.0, 297.5)` ).

## 4.2 Adaptive zoom (crop window)

**Function:** `adaptive_view_window()` — **our code decides zoom before AI sees anything.**

Uses distances to **nearby tags/symbols** in the tag register (valves, instruments, motors, peers — not the model):

```
Parent at (px, py)
→ measure distances to useful neighbors within ~230 CAD units
→ choose crop half-size (window radius)
```

| Layout                 | Signal                       | Zoom behaviour                               |
|------------------------|------------------------------|----------------------------------------------|
| Dense skid (pump area) | ≥6 neighbors within 75 units | Tighter crop (median distance × 1.85)        |
| Sparse vessel          | Fewer / farther neighbors    | Wider crop (78th percentile distance × 1.60) |
| Fallback               | No neighbors                 | Default ~137 units (between min/max)         |

Hard limits: **min ~105, max ~165** CAD units half-size (configurable via `--crop-half-min / --crop-half` ).

The window must also fit the **red parent highlight box** + ~45 unit margin, so the parent never gets clipped.

Center is biased: **55% parent XY + 45% highlight-box center** — keeps the machine in frame while showing attached lines.

### 4.3 Red PARENT box (separate from zoom)

**Function:** `parent_highlight_box()` + `overlay_parent_box()`

Two different rectangles:

| Rectangle   | Purpose                                                          |
|-------------|------------------------------------------------------------------|
| Crop window | What gets rasterized (includes neighbors, lines, peer equipment) |
| Red box     | Labels which symbol is the FUNCTION parent                       |

Red box logic:

- 1. Cluster same-tag register rows within ~55 CAD units (equipment + motor + local valves for 35-24L008 )
- 2. Include small equipment-layer INSERT bboxes near the parent
- 3. Pad ~24 units → draw thick red rectangle + label `PARENT 35-24L008` on the PNG

The model is told: *children belong under the red-box machine, not under distant peers.*

### 4.4 Render screenshot

**Function:** `viewer_screenshot()`

- Re-open DWG (ODA/ezdxf)
  - collect entities intersecting crop window
  - ezdxf Frontend + PyMuPDF backend → PNG @ 260 DPI
  - overlay red parent box
  - save outputs/evidence/Broke System.viewer\_<TAG>.png

Not a manual AutoCAD export — programmatic vector raster of the local window only.

### 4.5 Build text briefing (before vision call)

Two structured text blocks from Steps 1-3:

**CAD dossier** ( `build_equipment_dossier` , ~200 unit radius):

- Parent category / block / XY / descriptions
- Same-tag symbol family (equipment, motor, valves...)
- Nearby line short-ids + full pipe strings
- Nearby devices (valves, instruments, pumps)
- **Peer** primary equipment (e.g. adjacent pump — do not nest)
- Nearby drawing text tokens

**Candidate tags** ( collect\_candidate\_tags , ~220 unit radius):

- Whitelist of tag strings found near the crop
- Example: 35-24L008, 35-24P509, 35-24-009, 35-24-181, ...
- Model should prefer these over inventing tags

## 4.6 What goes to the vision model (Bedrock)

**Function:** bedrock\_hierarchy\_from\_shot() → single converse call:

```
content = [  
  { "text": <filled prompt ~6k chars> },  
  { "image": { "format": "png", "bytes": <viewer crop PNG> } }  
]
```

**Text prompt** ( prompts/pid\_hierarchy\_gt\_v4\_dossier.md ) includes:

| Section         | Example                                                   |
|-----------------|-----------------------------------------------------------|
| Plant context   | SITE, LINE, PROCESS, SUB-PROCESS=BR1                      |
| FUNCTION        | 35-24L008                                                 |
| CAD dossier     | Winder Pulper; lines 35-24-009, 35-24-011; peer 35-24P509 |
| Candidate tags  | Comma-separated whitelist                                 |
| Worked examples | Vessel vs pump hierarchy shape (teaching only)            |
| Rules           | Short line ids; peers ≠ children; JSON schema             |

**Image:** the adaptive crop with red PARENT highlight.

Vision models (Claude, Kimi, Qwen VL, etc.) get **both**. Text-only models (e.g. GPT-OSS) get prompt + dossier only, with a note that no image is available.

## 4.7 Model response → CSV

Model returns strict JSON:

```
{
  "function": "35-24L008",
  "rows": [
    {"equipment": "35-24-009", "subequipment": ""},
    {"equipment": "", "subequipment": "35-24-1088"}
  ],
  "peers": [{"tag": "35-24P509", "evidence": "adjacent pump"}]
}
```

Post-processing ( refine\_ai\_hierarchy ) trims invalid tags, then rows flatten into:

SITE → LINE → PROCESS → SUB-PROCESS → FUNCTION → EQUIPMENT → SUB-EQUIPMENT

### Files written:

| File                                           | Purpose                        |
|------------------------------------------------|--------------------------------|
| outputs/Broke System.hierarchy.csv             | Full hierarchy sheet           |
| outputs/Broke System.hierarchy_gt.csv          | GT-shaped columns for scoring  |
| outputs/jsons/Broke System.hierarchy_ai.json   | Raw model responses + dossier  |
| outputs/evidence/Broke System.viewer_<TAG>.png | Adaptive crop + red PARENT box |
| outputs/logs/hierarchy-ai.log                  | Run log                        |

## Sample output

**Viewer screenshot** (what the vision model sees):

```
outputs/evidence/Broke System.viewer_35-24L008.png
# adaptive crop around Winder Pulper + red box labelled "PARENT 35-24L008"
```

**CAD dossier** (text injected into prompt):

Parent tag: 35-24L008  
Parent category/block: process\_equipment / PPI\_1302A-25\_0  
Parent XY: (660.0, 297.5)  
Nearby descriptions: WINDER PULPER; HP-33G2; 800 BDTPD

Same-tag CAD family:  
- process\_equipment @ (660.0,297.5) block=PPI\_1302A-25\_0 nearby=35-24L008; 35-24P509  
- motors @ (680.0,232.5) block=PPI\_1504A-25\_0 nearby=35-24L008; 35-24P509

Nearby line numbers (short ids preferred in output):  
- 35-24-009 full=35-24-009-PP-600-E10H2A d=46 layer=P-LINEPOS  
- 35-24-011 full=35-24-011-PP-250-E10H2A d=107 layer=P-LINEPOS

### Candidate tags (whitelist near crop):

35-24L008, 35-24P509, 35-24-009, 35-24-181, 35-24-226, 35-24-1088, ...

### Model JSON response (from hierarchy\_ai.json):

```
{
  "function": "35-24L008",
  "sub_process": "BR1",
  "rows": [
    {"equipment": "35-24-009", "subequipment": "", "mask": ""},
    {"equipment": "35-24-008.1", "subequipment": "", "mask": ""},
    {"equipment": "", "subequipment": "35-24-181", "mask": ""}
  ],
  "peers": [{"tag": "35-24P509", "evidence": "adjacent pump, not child"}]
}
```

### Hierarchy CSV (flattened sheet rows):

```
SUB-PROCESS,FUNCTION,EQUIPMENT,SUB-EQUIPMENT,MASK
BR1,35-24L008,,,SHOTTONPM3-BROKESYSTEM-BR1-35-
,,35-24-008.1,,35-24-008.1
,,35-24-008.2,,35-24-008.2
,,35-24-009,,35-24-009
,,,35-24-181,35-24-181
```

# Step 4 flow (one tag)

- 35-24L008
- └ parent XY from tag register
  - └ adaptive crop window (from nearby tag distances)
  - └ red parent box on raster PNG
  - └ dossier + candidates from JSON caches
  - └ prompt + PNG → Bedrock
  - └ JSON rows → hierarchy CSV

## How steps connect

| Step        | Input                        | Output                            | AI?                  |
|-------------|------------------------------|-----------------------------------|----------------------|
| 1 Read DWG  | .dwg                         | structural_dump.json              | No                   |
| 2 Tags+XYZ  | structural + layers          | pid_inventory.json , tag register | No                   |
| 3 Enrich    | inventory + structural       | pid_enrichment.json               | No                   |
| 4 Hierarchy | enrichment + DWG + tags list | CSV + evidence PNGs               | Yes (Bedrock vision) |

## Commands

```
make run-json      # Step 1: DWG → structural_dump.json
make inventory     # Step 2a: symbols + XYZ
make enrich       # Step 2b + Step 3: tag register + line bindings + title block
make all          # Steps 1-3

make hierarchy-ai TAGS=35-24L008,35-24P509 # Step 4
make hierarchy-eval                               # score vs GT
```

Verify ODA before first run:

```
make check-oda-fc
```

Debug screenshots only (no Bedrock):

```
python3 dwg_pid_hierarchy_ai.py --shots-only --tags 35-24L008
```

## Design choices (why it works this way)

1. **Layer-first classification** — matches how this P&ID was authored; more reliable than block name alone.
2. **Spatial tag binding** — plant tags are text annotations, not block attributes; nearest-neighbour within ~90 units is the correct model.
3. **JSON cache between stages** — dump once, reuse for inventory/enrich/AI without re-opening DWG.
4. **Adaptive zoom from CAD distances** — crop size follows local density of nearby symbols; AI does not choose the camera.
5. **Red box ≠ crop box** — wide context in the image, but explicit parent marker so ownership is unambiguous.
6. **Dossier + candidates + image** — vision models read the drawing; text briefing grounds them in real tags and reduces invention.
7. **Preserve Z** — P&ID is 2D, but we keep full insert points for consistency with CAD exports.

## Related docs

- Full pipeline + model scores: `FULL_PIPELINE_RUNDOWN.md`
- Screenshot zoom + AI prompt packet: `HIERARCHY_AI_PROMPT_RUNDOWN.md`